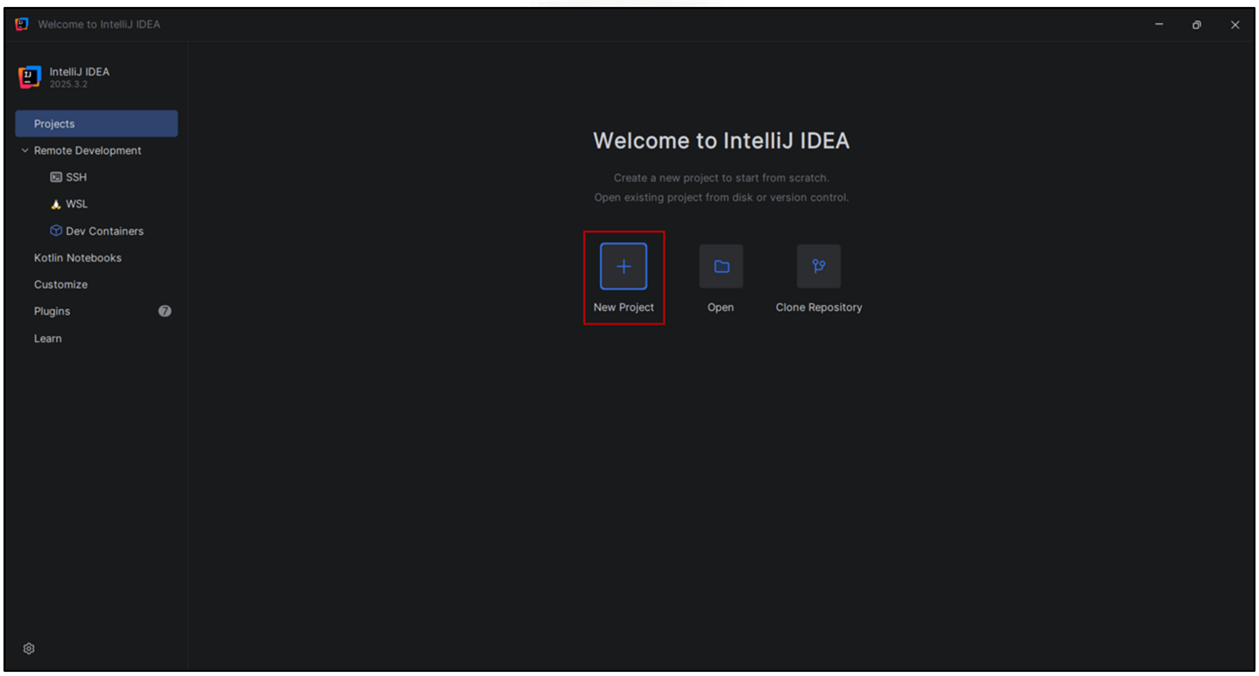
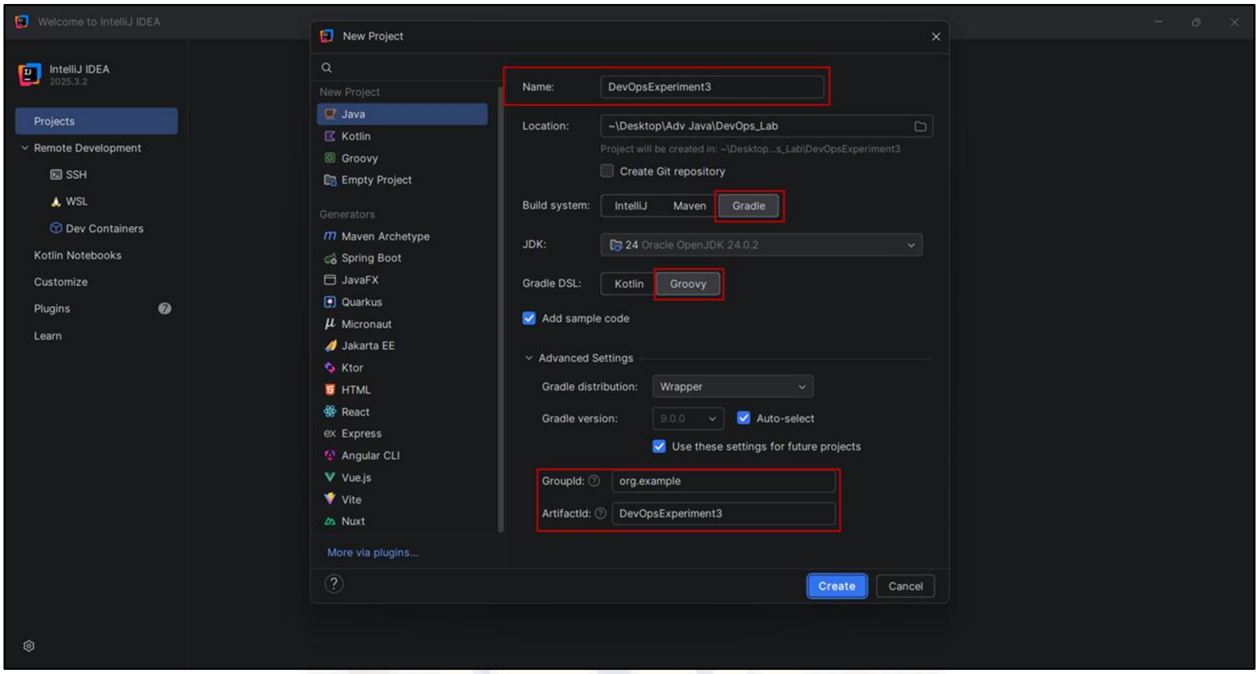
DevOps Lab Experiment 3

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

Step 1: Create a New Gradle Project in **IntelliJ IDEA**

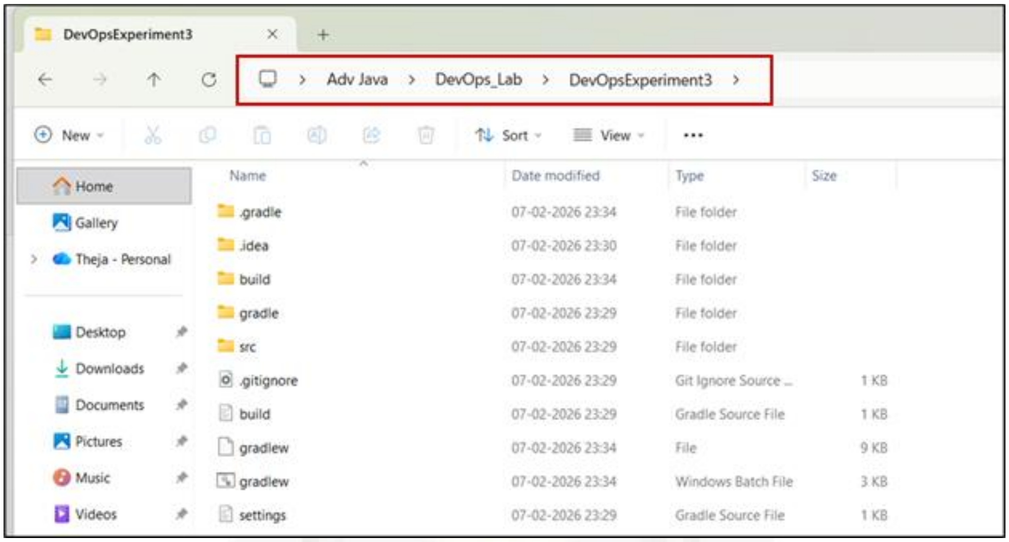
Open Intelli IDEA and click **New Project**.





1. From the left panel, select **Java**
2. Enter the Project Name as: **DevOpsExperiment3**
3. Select the **Location** to save the project
4. Under Build system, select **Gradle**
5. Under Gradle DSL, select **Groovy**
6. Select the required **JDK version**
7. Keep the GroupId as: **org.example**
8. Keep the ArtifactId as: **DevOpsExperiment3** *(Name and ArtifactId should be same)*
9. Click **Create.**

IntelliJ will **create a new Gradle-based Java project folder** in the selected location.

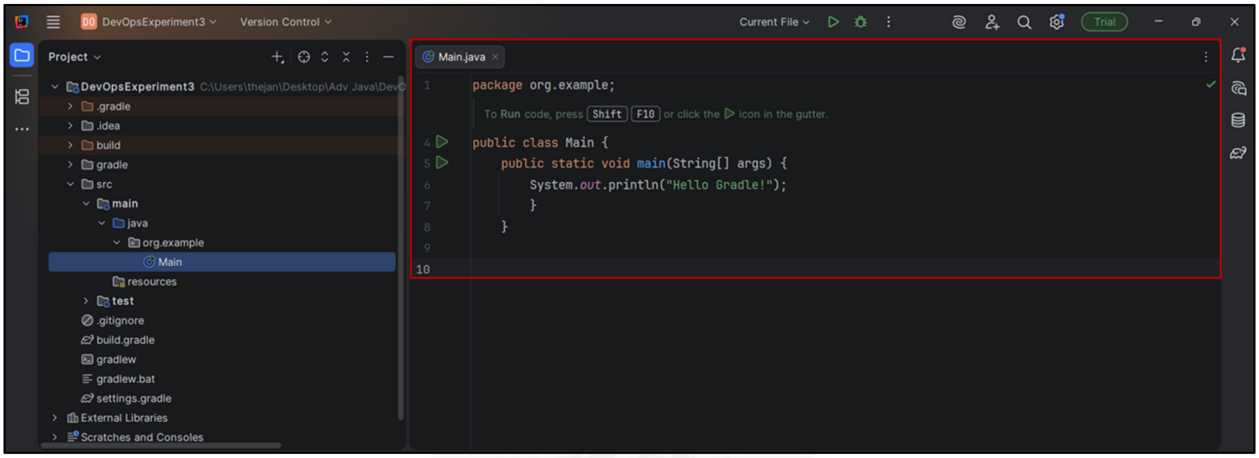


Step 2: Create a Main Java file

If ***Add sample code*** is selected during project creation, the Main.java file will be created automatically. Otherwise, we must create it manually.

To create a Main Java class, right-click on **org.example**, select **New** → **Java Class**, and name the class as **Main**. If it already exists, open the existing file.

```
package org.example;
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Gradle!");
    }
}
```

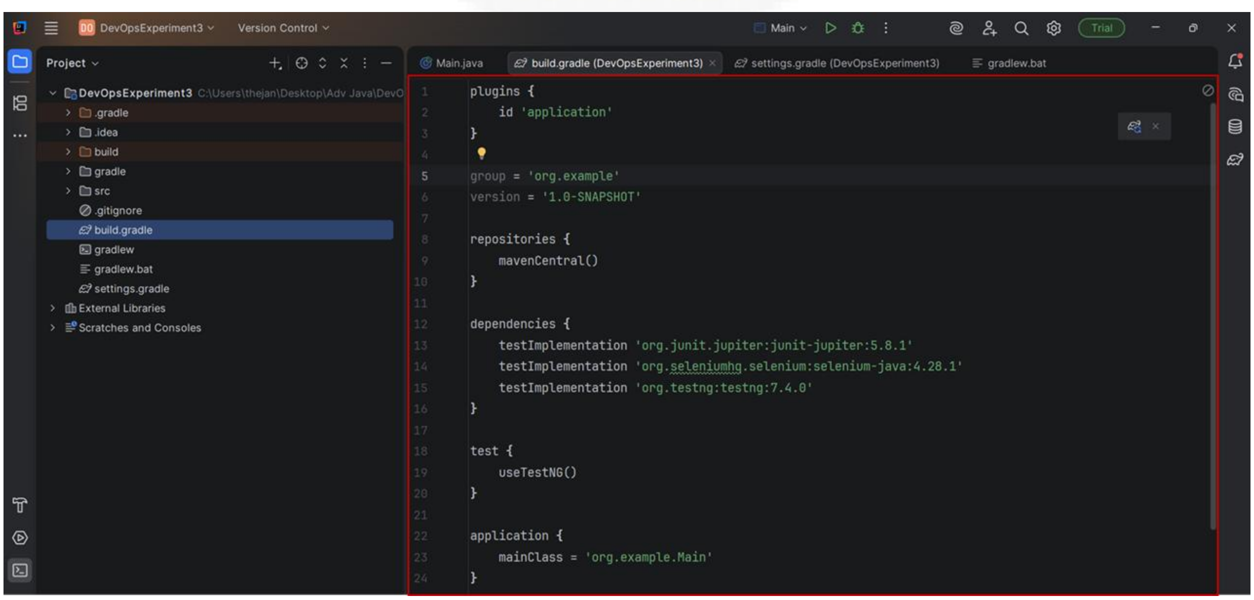


Main.java file

Step 3: Update build.gradle file

Open the build.gradle file inside the project folder and replace its contents with the following code.

```
plugins {
    id 'application'
}

group = 'org.example'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'
    testImplementation 'org.seleniumhq.selenium:selenium-java:4.28.1'
    testImplementation 'org.testng:testng:7.4.0'
}

test {
    useTestNG()
}

application {
    mainClass = 'org.example.Main'
}
```



build.gradle

Step 4: Run the project.

Open the Terminal in **IntelliJ IDEA** and run the command gradle run.

```
Terminal   Local
PS C:\Users\thejan\Desktop\Adv Java\DevOps_Lab\DevOpsExperiment3> gradle run

> Task :run
Hello Gradle!

BUILD SUCCESSFUL in 2s
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html
PS C:\Users\thejan\Desktop\Adv Java\DevOps_Lab\DevOpsExperiment3>
```

Step 5: Hosting Static Website on GitHub Pages

1. Create a **docs** folder inside the **Gradle project folder** (Here, the project folder is DevOpsExperiment3).

In the left side of Project panel of IntelliJ IDEA, Right click on **DevOpsExperiment3 → New → Directory**. Name the directory as **docs**.

2. **Create index.html**

Right click on docs → New → HTML File, name the file as **index.html** and paste the given html code.

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Tripillar Solutions</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
   <div class="container">
      <img src="tripillar.jpg" alt="Logo" class="logo">
      <h1>Tripillar Solutions</h1>
      <p>Welcome to our official website</p>
   </div>
</body>
</html>
```

3. **Create style.css**

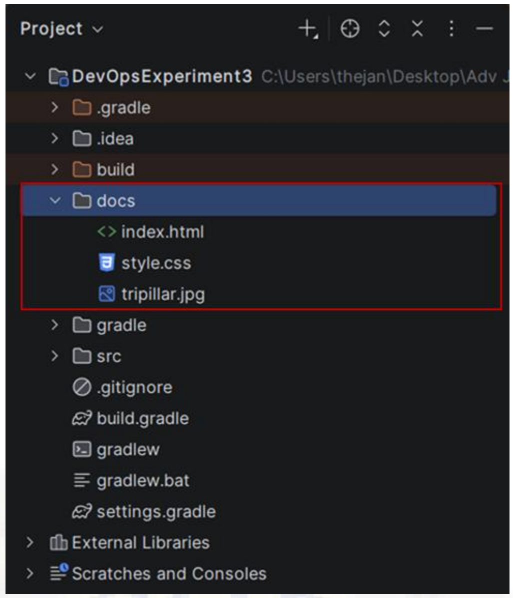
(Right click on docs → New → Stylesheet -> CSS file, name the file as style.css and paste the given CSS code.

```
body {
   font-family: Arial, sans-serif;
   background-color: #f2f2f2;
}
.container {
   text-align: center;
   margin-top: 100px;
}
.logo {
   width: 150px;
   height: auto;
}
h1 {
   color: #2c3e50;
}
p {
   color: #555;
   font-size: 18px;
}
```

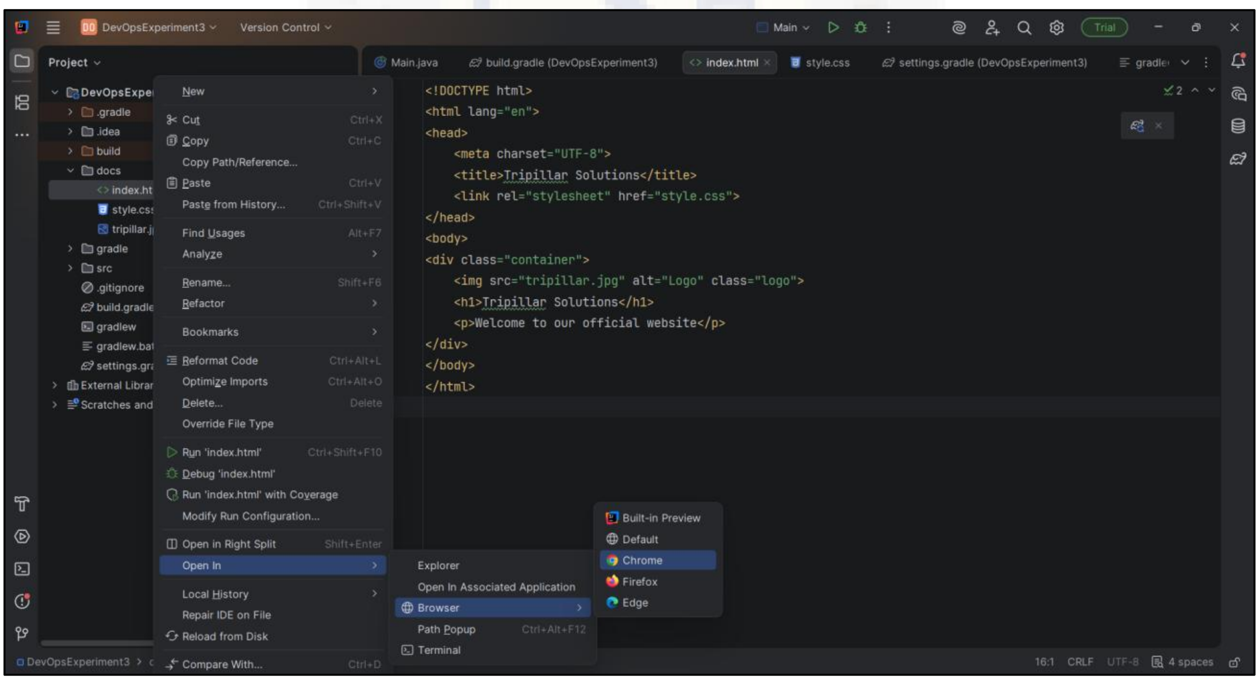
4. Copy an image file into the docs folder and name it as **tripillar.jpg**. (https://tinyurl.com/murj2ap6)

   Now the **docs** folder should contain: **index.html, style.css, and the image file**.

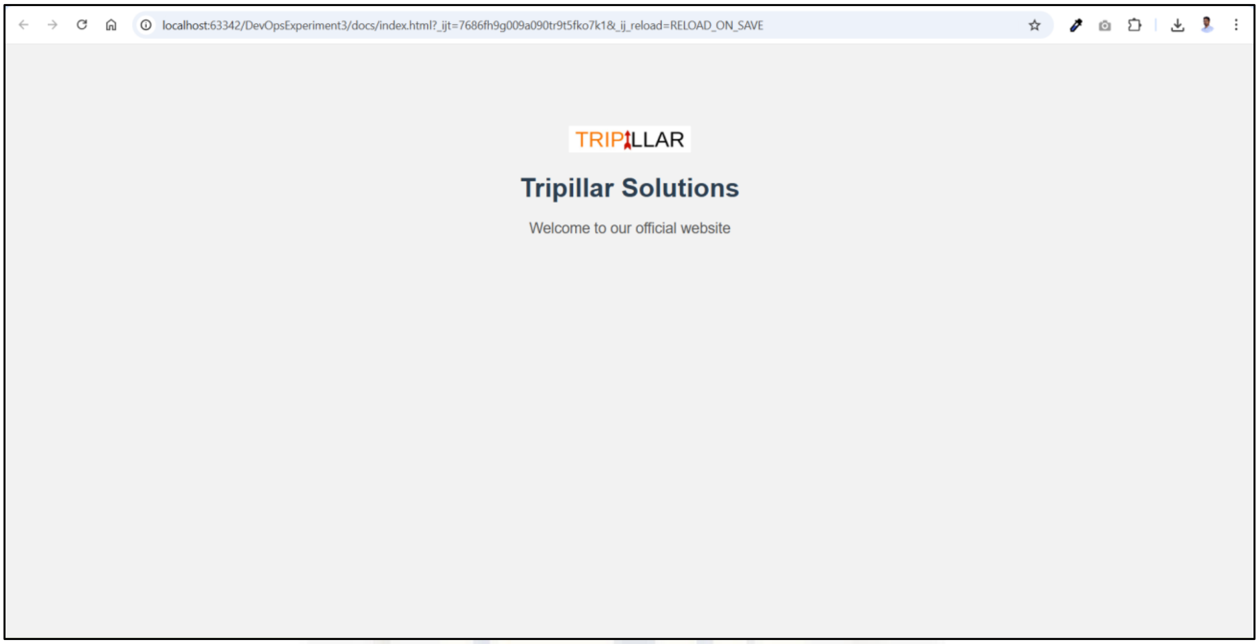
5. Right-click index.html inside the **docs folder** → Open in → Browser → Chrome and observe that the web page is rendered in the browser.



docs folder contents



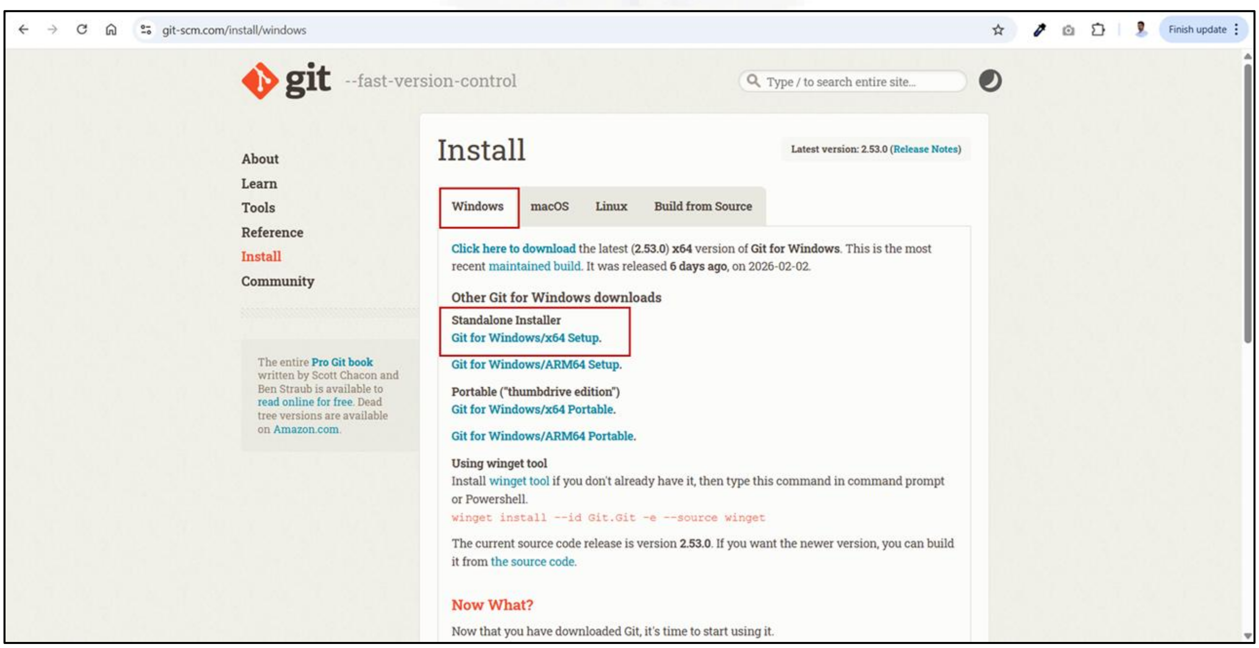
Opening index.html in Chrome



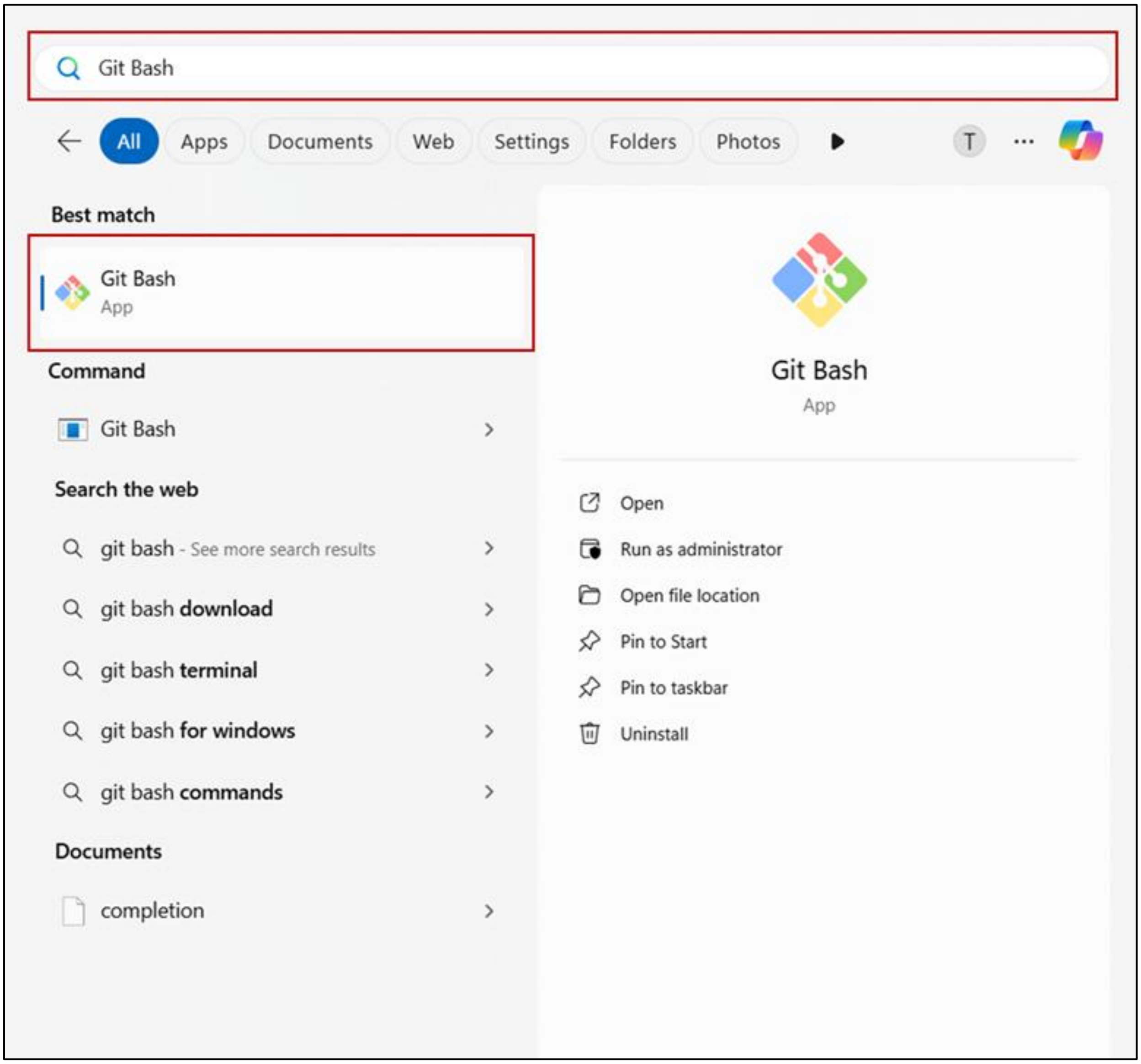
index.html rendered in Chrome browser

Step 6: Git installation and GitHub Account Creation

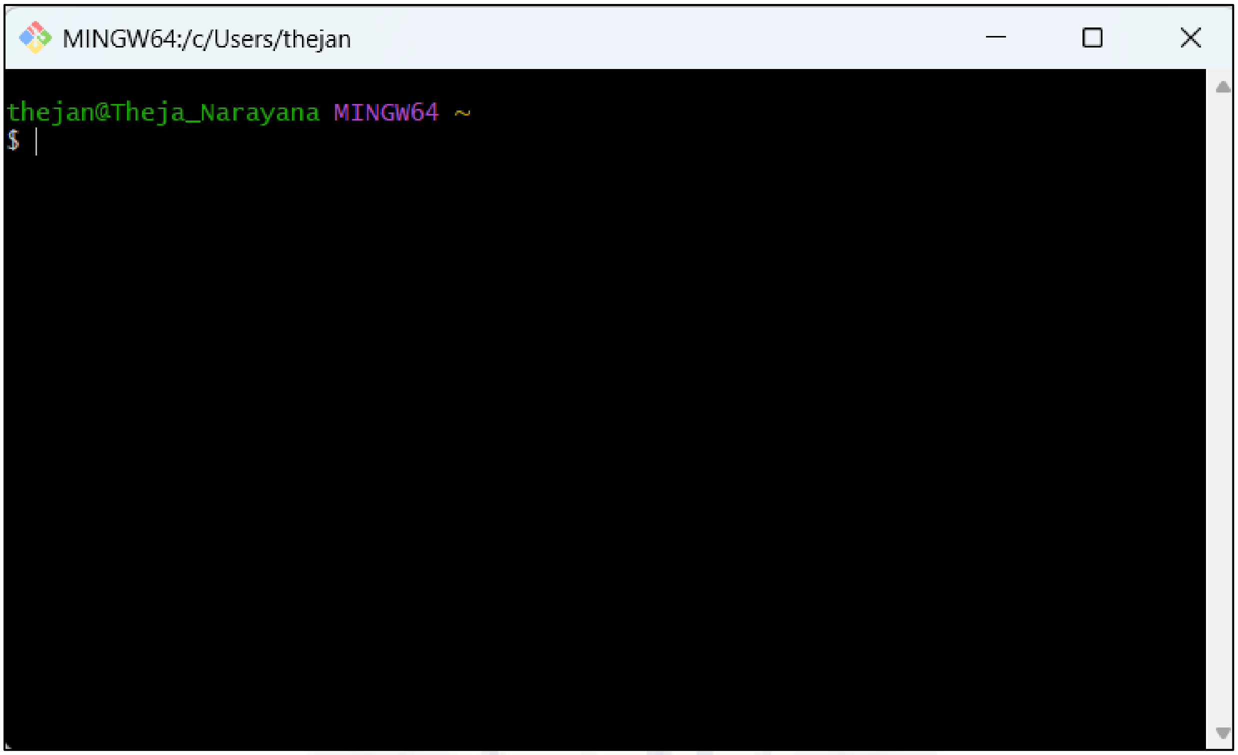
1. Go to https://git-scm.com/install/windows, download and install git.



2. Open **Start** menu, type Git Bash, and open it. The **Git Bash** is installed along with Git, and it provides a Linux-like terminal to run Git commands.

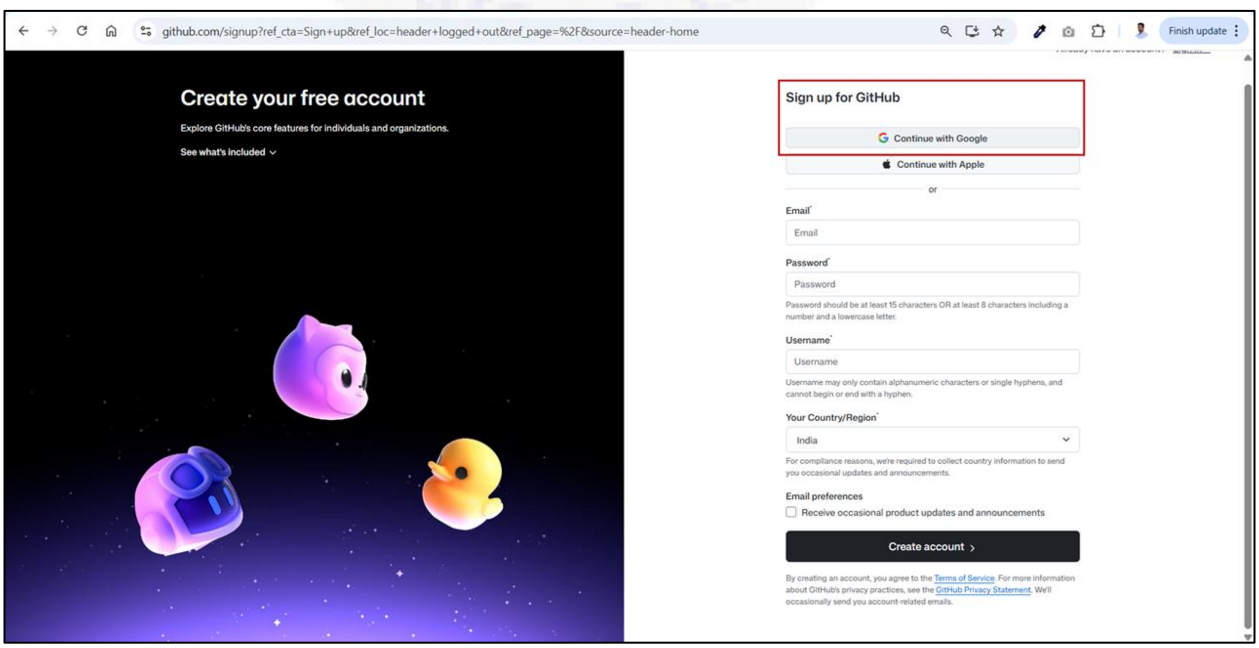


Searching for Git Bash in the Start Menu



Git Bash terminal opened successfully on Windows

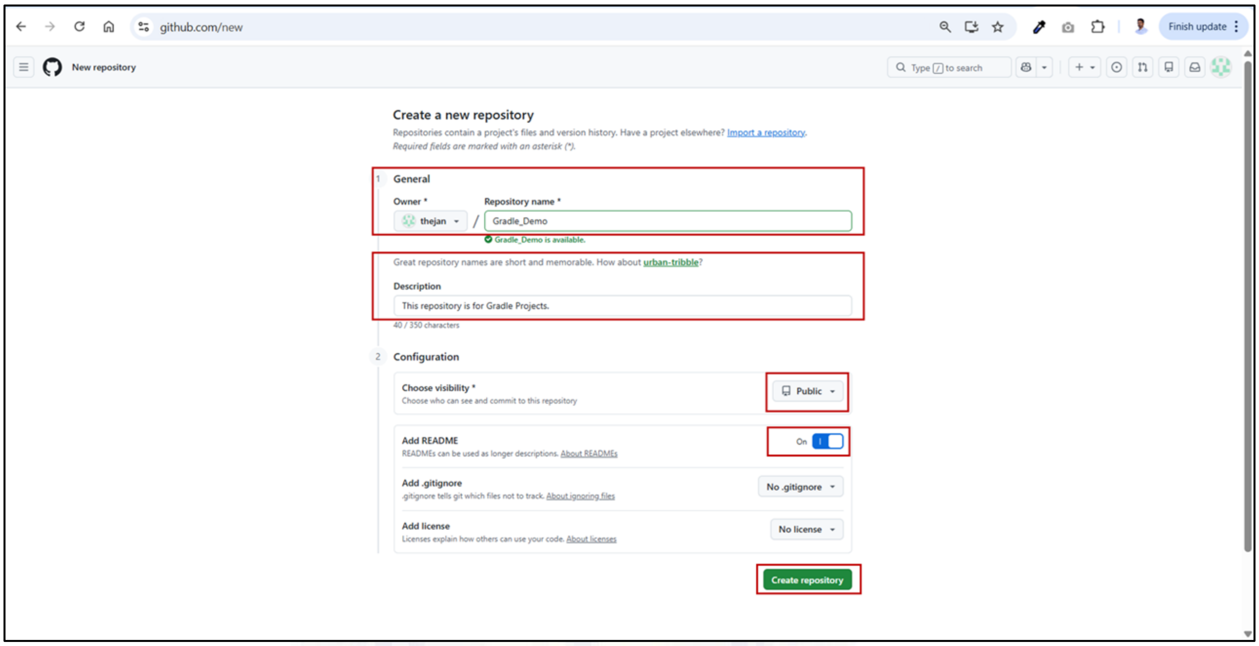
3. Go to https://github.com/, Sign up for GitHub account using Gmail account.



Sign up for GitHub

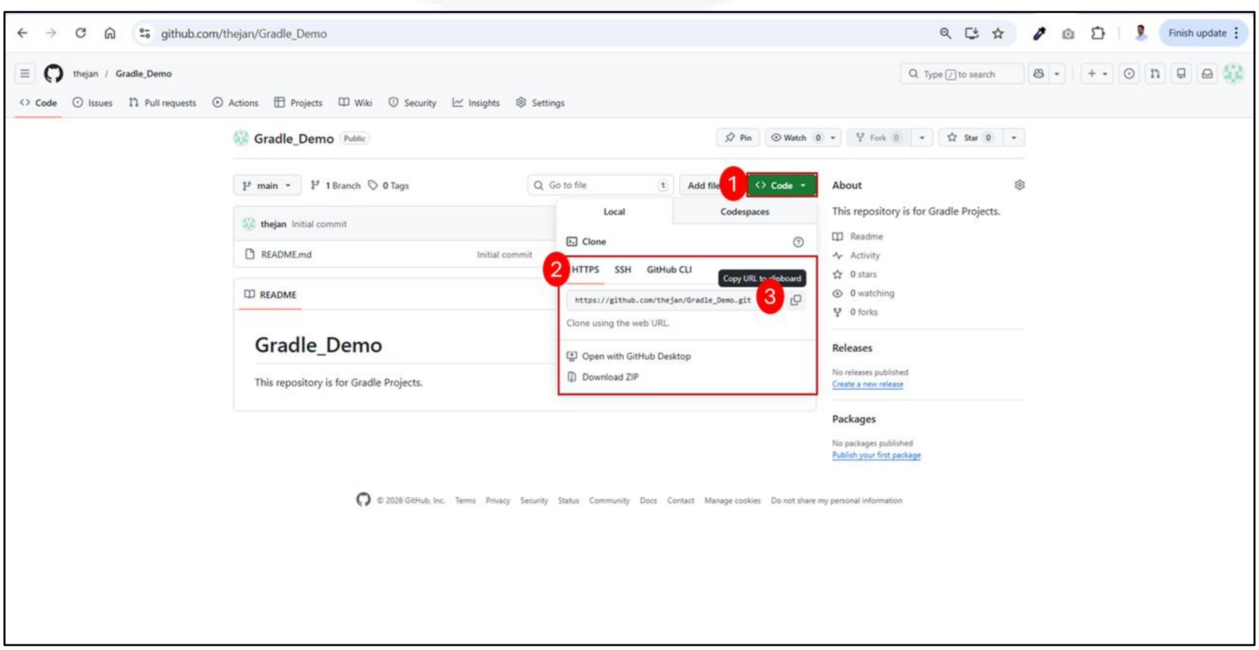
Step 7: **Create a Git repository**

1. On the GitHub Dashboard, click the **user navigation menu** in the top-right corner and select **Repositories**. Then click **New**, enter the **Repository name** and **Description** (optional), and click **Create repository**.

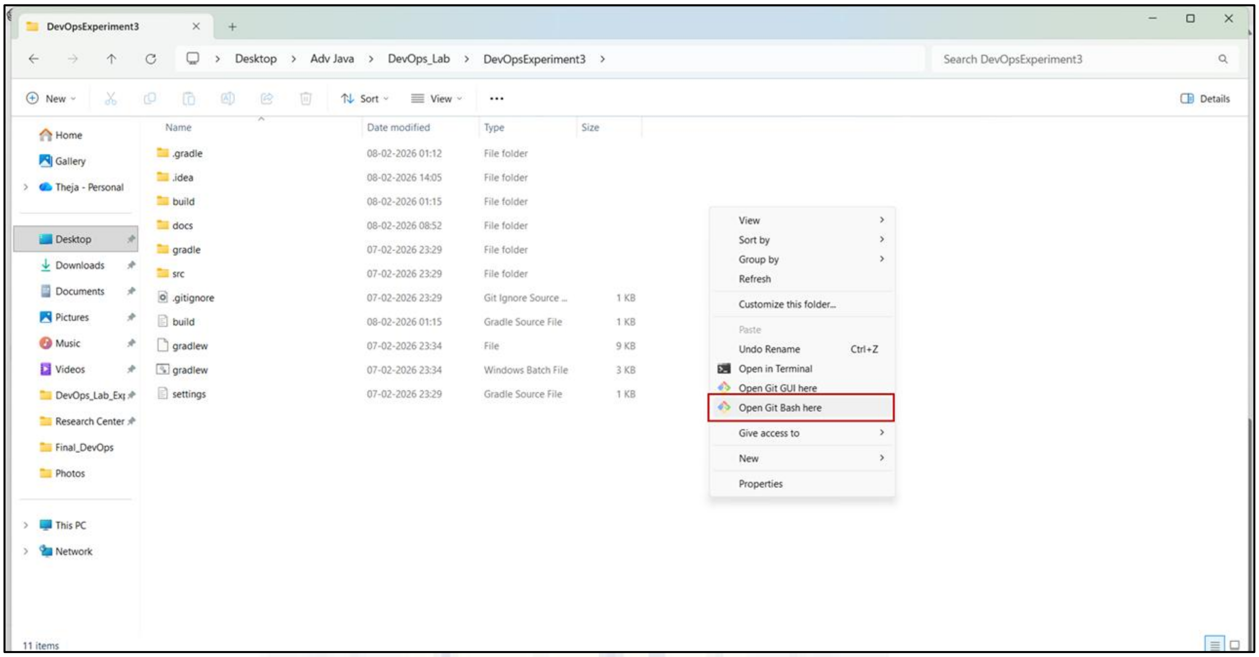


Creating a new Repository

2. Click the Code button and copy the **HTTPS repository URL.**



3. Navigate to the project folder on your system, right-click on an empty space, select Show more options, and then click Open **Git Bash here**.



4. Type and execute the following commands in the Git Bash terminal.

git init

git status

git add .

git commit -m "Added website files for GitHub Pages"

git remote add origin url

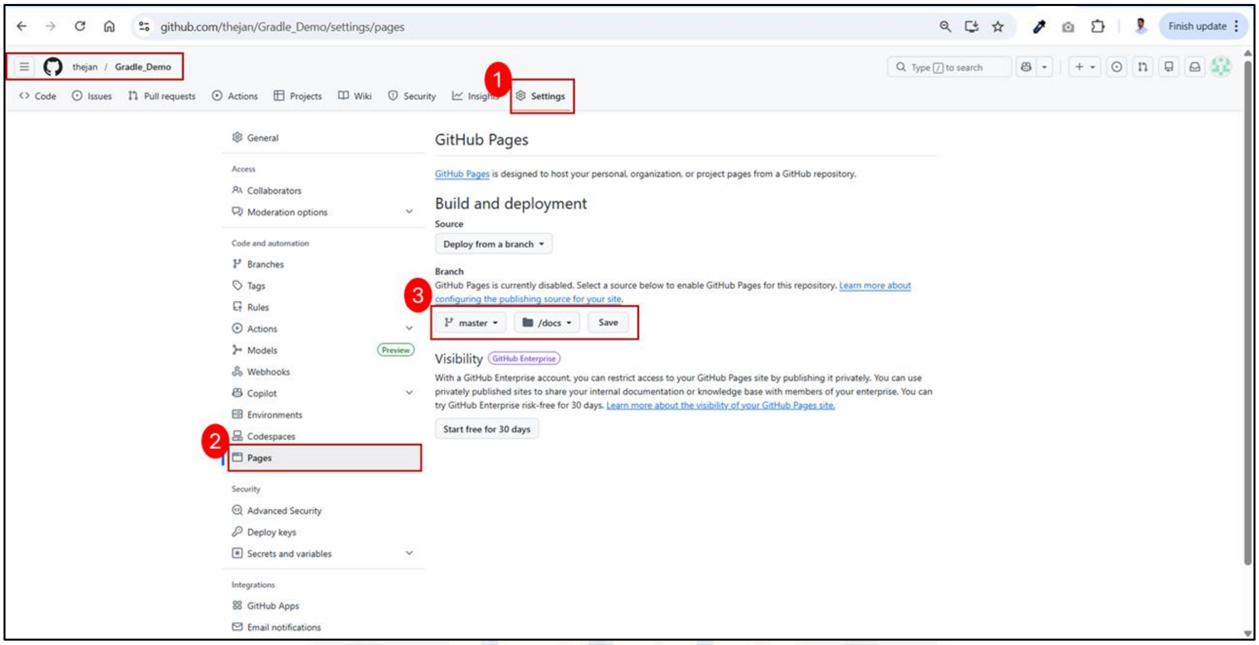
(Replace url by the HTTPS repository URL)
Example : git remote add origin https://github.com/thejan/Gradle_Demo.git

git push -u origin master

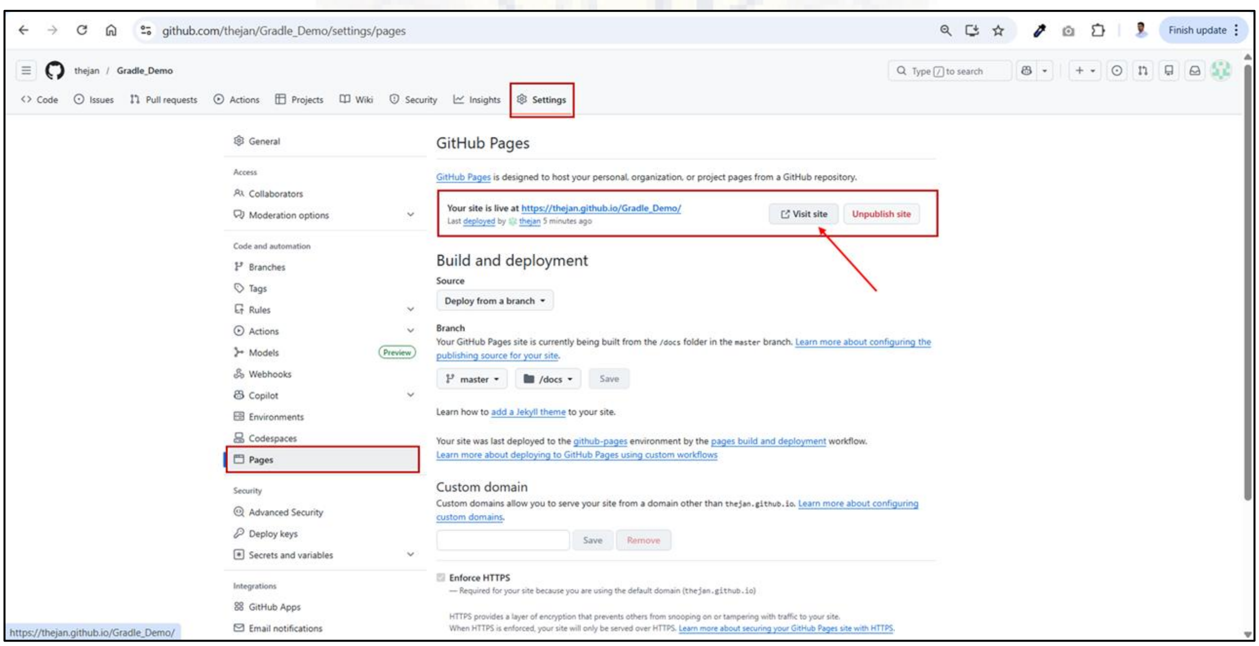
```
MINGW64:/c/Users/thejan/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3
$ git init
Initialized empty Git repository in C:/Users/thejan/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3/.git/

thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        .idea/
        build.gradle
        docs/
        gradle/
        gradlew
        gradlew.bat
        settings.gradle
        src/

nothing added to commit but untracked files present (use "git add" to track)

thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'build.gradle', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'gradle/wrapper/gradle-wrapper.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'gradlew', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/org/example/Main.java', LF will be replaced by CRLF the next time Git touches it

thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ git commit -m "Added website files for GitHub Pages"
[master (root-commit) 5a6cc79] Added website files for GitHub Pages
 14 files changed, 502 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 .idea/.gitignore
 create mode 100644 .idea/gradle.xml
 create mode 100644 .idea/misc.xml
 create mode 100644 build.gradle
 create mode 100644 docs/index.html
 create mode 100644 docs/style.css
 create mode 100644 docs/tripillar.jpg
 create mode 100644 gradle/wrapper/gradle-wrapper.jar
 create mode 100644 gradle/wrapper/gradle-wrapper.properties
 create mode 100644 gradlew
 create mode 100644 gradlew.bat
 create mode 100644 settings.gradle
 create mode 100644 src/main/java/org/example/Main.java

thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ |
```

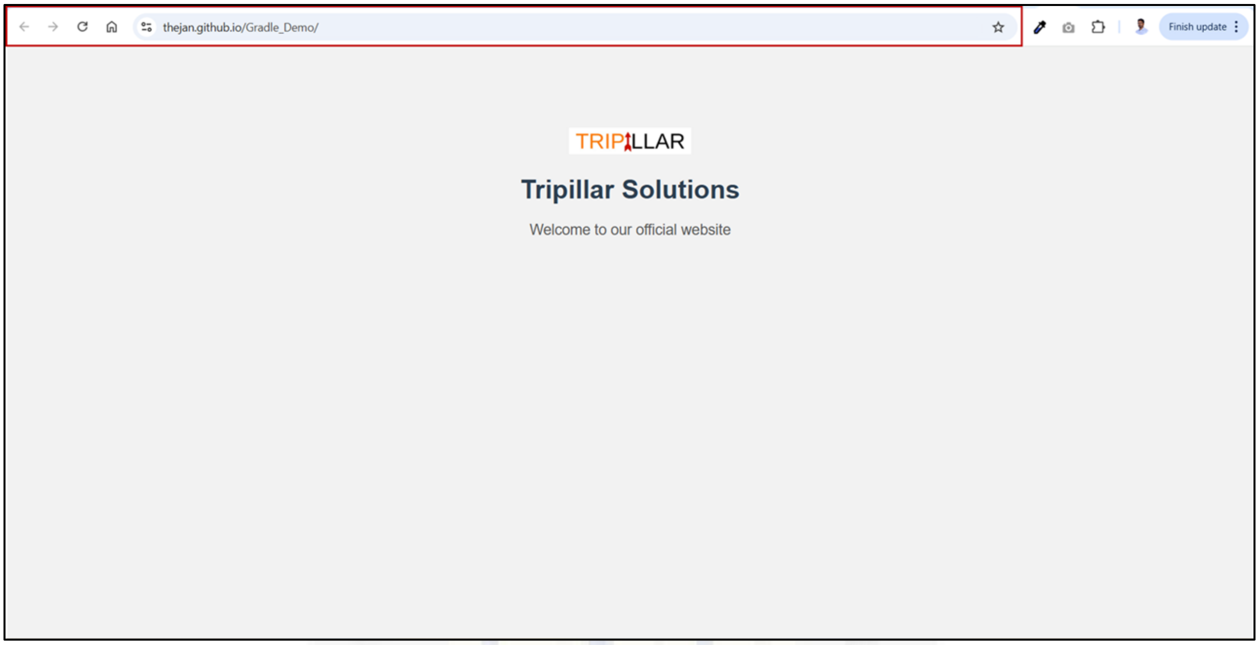
```
thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ git remote add origin https://github.com/thejan/Gradle_Demo.git

thejan@Theja_Narayana MINGW64 ~/Desktop/Adv Java/DevOps_Lab/DevOpsExperiment3 (master)
$ git push -u origin master
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 4 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (25/25), 52.10 KiB | 5.79 MiB/s, done.
Total 25 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/thejan/Gradle_Demo/pull/new/master
remote:
To https://github.com/thejan/Gradle_Demo.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

5. Go to the **Gradle Demo repository on GitHub**, click **Settings → Pages**, select the **Branch** and folder under **Build and deployment**, and then click **Save**.



6. After enabling GitHub Pages, refresh the page, then click the URL shown under "Your site is live at …" (or click Visit site) to open the hosted website.

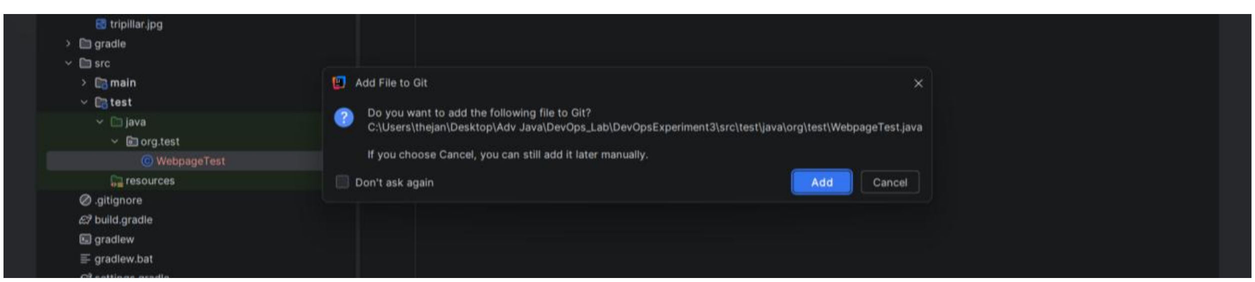




Hosted Website using GitHub Pages

Step 8: Automation Testing of Published Website

- In IntelliJ IDEA Project panel, right click on **src** → **New** → **Directory** → **type: test/java/org/test**
- Right click on **org.test** → **New** → **Java Class** → type **WebpageTest** and paste the WebpageTest.java code
- Click on **Add** button to add the test file to Git
- Update the URL in the code with **your GitHub Pages website URL**
  Example : https://thejan.github.io/Gradle_Demo/
- **Run the Automation Test**:
  In the IntelliJ IDEA terminal, execute: **gradle test**. This will launch the browser and open the website.



Adding test file to Git

```java
package org.test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

public class WebpageTest {

    WebDriver driver;

    @BeforeTest
    public void openBrowser() throws InterruptedException {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        Thread.sleep(2000);
        driver.get("https://thejan.github.io/Gradle_Demo/");
    }

    @Test
    public void titleValidationTest() {
        String actualTitle = driver.getTitle();
        String expectedTitle = "Tripillar Solutions";
        Assert.assertEquals(actualTitle, expectedTitle);
    }

    @AfterTest
    public void closeBrowser() throws InterruptedException {
        Thread.sleep(1000);
        driver.quit();
    }
}
```

WebpageTest.java file contents

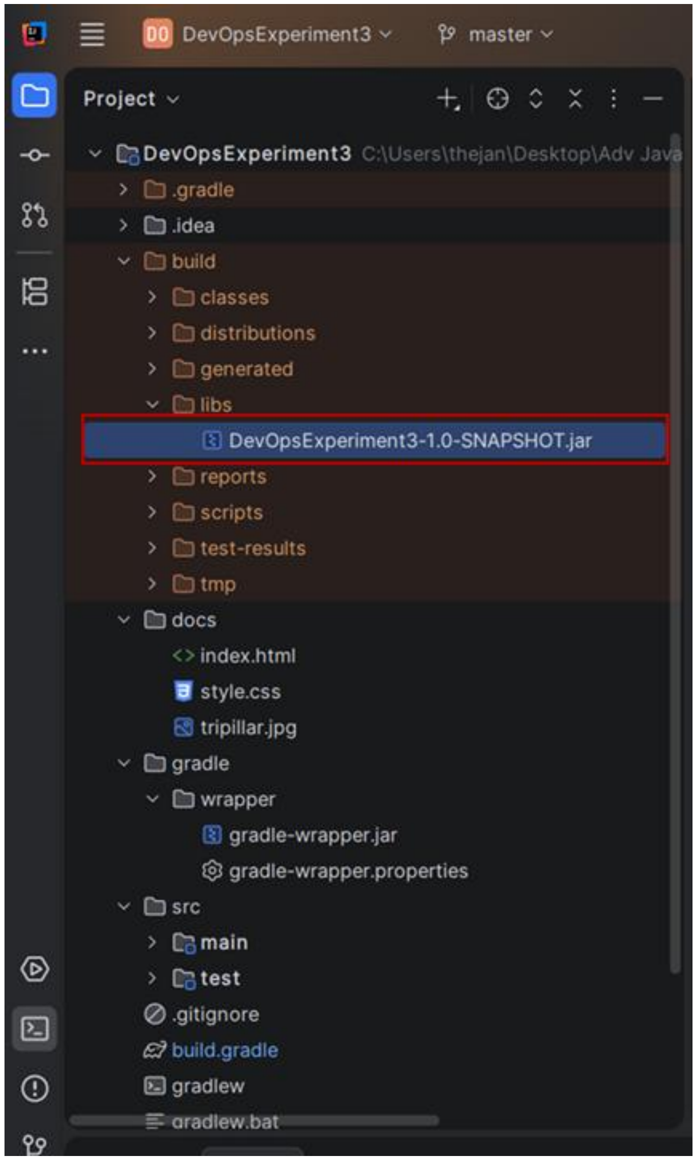
Step 9: Create JAR file

Update the build.gradle with the following code

```
plugins {
   id 'application'
}
group = 'org.example'
version = '1.0-SNAPSHOT'
repositories {
   mavenCentral()
}
dependencies {
   testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'
   testImplementation 'org.seleniumhq.selenium:selenium-java:4.28.1'
   testImplementation 'org.testng:testng:7.4.0'
}
test {
   useTestNG()
}
application {
   mainClass = 'org.example.Main'
}

jar {
   manifest {
      attributes 'Main-Class': 'org.example.Main'
   }
}
```

In the IntelliJ IDEA terminal, execute the following

- **./gradlew build**
- **./gradlew jar**



jar is created inside build/libs folder

- **Run the JAR file**
  java -jar build/libs/DevOpsExperiment3-1.0-SNAPSHOT.jar

VIVA QUESTIONS

1. What is Gradle?
2. Why do we use Gradle in DevOps?
3. What is Build Automation?
4. What is the purpose of build.gradle file?
5. What is Gradle DSL?
6. What is the difference between Groovy DSL and Kotlin DSL?
7. What is the use of plugins block in Gradle?
8. What is the use of id 'application' plugin?
9. What is the purpose of repositories { mavenCentral() }?
10. What is dependency management in Gradle?
11. What is the difference between implementation and testImplementation?
12. What is the use of gradle run command?
13. What is the use of ./gradlew build command?
14. What is the use of ./gradlew jar command?
15. What is Gradle Wrapper and why is it used?
16. What is the purpose of .gradle folder?
17. What is a JAR file?
18. Where is the JAR file generated in Gradle project?
19. What is the use of command java -jar filename.jar?
20. What is JAVA_HOME and why is it important?
21. What is Git and why is it used?
22. What is GitHub Pages?
23. What is a static website?
24. What is Selenium WebDriver?
25. What is the purpose of TestNG in automation testing?